Abstract

This document presents the design, implementation, and evaluation of a comprehensive booking management system (for Express BnB) developed in Java. The system facilitates seamless interaction between users, such as administrators, guests and hosts, to manage properties, accommodations, and bookings effectively.

With an emphasis on user-friendly design, advanced functionality, and adherence to object-oriented programming principles, the application provides robust features such as booking creation, cancellation, review management, notification handling and analytics. Additionally, the application leverages Java Streams and advanced design patterns to optimize data processing and system scalability.

The report is structured to provide an in-depth exploration of the application's development lifecycle, from requirements analysis to final implementation, including GUI design, testing, and documentation. Key considerations, such as error handling, security, and usability, are highlighted to demonstrate the project's practicality and extensibility. Concluding with future work and potential enhancements, and will serve as a comprehensive guide to understanding the development and functionality of the booking management system.

Methodology

Development Environment

The development of Express BnB utilized a modern Java-based technology stack to ensure scalability, maintainability, and robust functionality. Java served as the primary programming language, chosen for its object-oriented programming (OOP) strengths, which allowed for a modular and reusable architecture. The GUI was implemented using JavaFX, enabling the creation of an interactive and visually engaging user interface. Additionally, Java Streams were employed for efficient data manipulation, enhancing the application's performance and simplifying complex workflows.

Object Creation and Main Class Implementation

At the heart of the application lies the main class, which initializes core components, orchestrates state transitions, and launches the GUI. During system initialization, essential objects such as properties, accommodations, users, and bookings are instantiated, setting the foundation for a fully functional platform. Dynamic object creation was a critical feature, allowing the application to respond to user actions in real-time. For example, when a guest creates a booking, a new instance of the bookingInfo class is dynamically generated, transitioning through states like "Pending" or "Confirmed." This behaviour is underpinned by the State Design Pattern, which provides seamless adaptability to changes in object behaviour based on current states.

Java Streams Integration

Java Streams played a pivotal role in simplifying data handling and enhancing performance across the application. Key use cases include:

- **Filtering Accommodations**: Streams enabled dynamic filtering of accommodations based on user-defined criteria such as location, price range, and amenities.
- **Retrieving Pending Bookings**: Hosts could effortlessly retrieve bookings with a "Pending" status using concise stream-based filtering, reducing code complexity.
- **Data Aggregation**: Streams streamlined the calculation of metrics, such as total revenue or available accommodations, improving the application's analytical capabilities.
- **User Role Validation**: By leveraging streams, the application dynamically validated user roles, ensuring appropriate access control for administrators, hosts, and guests.
- **Enhanced GUI Updates**: Real-time responsiveness was achieved by dynamically updating JavaFX components, such as ListView and ChoiceBox, with stream-processed data.

User-Friendly Outputs

Ensuring intuitive and clear outputs was a cornerstone of the application's design. Booking details were formatted and displayed using stream-based processing, presenting information concisely to users. For example, the application could generate a summary of bookings, including IDs, accommodation names, and statuses, through streamlined formatting methods. This functionality seamlessly integrated with JavaFX elements, providing a dynamic and engaging user experience.

Error Handling and Security Measures

Robust error handling and security measures were implemented to guarantee reliability and user satisfaction. Streams were employed for efficient validation of data, ensuring that only accurate and relevant information was processed. Role-based access control was enforced, enhancing the application's security by restricting functionality based on user roles. Additionally, user feedback was prioritized, with clear and informative error messages guiding users through any encountered issues.

The combination of a robust development environment, dynamic object creation, efficient Java Streams integration, user-friendly outputs, and stringent error handling ensured the development of a reliable, scalable, and user-centric accommodation booking platform.

Introduction

The Express BnB application is designed as a robust and flexible accommodation booking platform, enabling users to explore and reserve a variety of lodgings tailored to different preferences and budgets. In addition to featuring options such as sofas, luxurious villas, flats and hotels, Express BnB accommodates diverse user needs by offering facilities such as kitchens, washing machines, and other amenities. Leveraging Java to provide a scalable and object-oriented architecture; ensuring high performance and maintainability.

To achieve a well-structured and modular design, Express BnB is modelled after the Layered Structure Operating System (OS) Model, a concept borrowed from operating systems design & architecture. This approach introduces a unique layered architecture, where each layer interacts with its adjacent layers to manage complexity and maintain separation of concerns.

For example, properties can contain multiple accommodations, and accommodations dynamically transition between booked and not-booked states, influenced by a nested state pattern handling availability. This design ensures that both properties and accommodations remain synchronized with their current states.

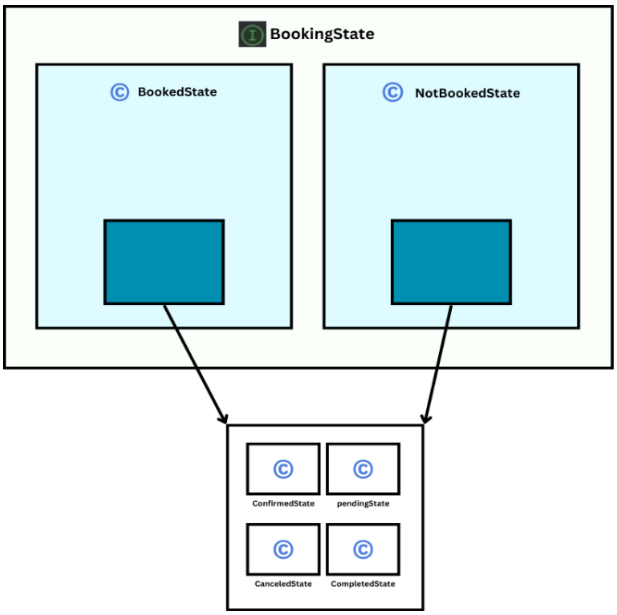


Figure.1 illustrating the Visualization of the Implemented Layered State Pattern

Unified Modelling Language (UML) Integration

UML provides a standardized way to visualize and communicate the architecture and behaviours of the application, making it easier to understand relationships and interactions between components.[1]

For Express BnB, UML diagrams such as class diagrams, sequence diagrams, and use case diagrams will serve as a blueprint which visualizes the layered state pattern, user interactions, and system structure.

The layered state pattern is a unique aspect of the system, integrating two levels of state management:

1. **Accommodation State Management**: Accommodations transition between Booked and NotBooked states.
2. **Availability State Management**: Within each accommodation, availability is further managed by a secondary state pattern with four states (Pending, Confirmed, Completed, and Canceled). This nested state influences the overall property state, ensuring synchronization across the system.

JavaFX: The Chosen Framework for GUI Development

To deliver an intuitive and responsive user interface, Express BnB is built using JavaFX, a powerful and versatile Java framework for building rich client applications. JavaFX is chosen for its ability to:

- Create dynamic and visually appealing interfaces with features like animations and custom styling.[2]
- Simplify the development process with its FXML markup for defining the user interface. [2]
- Support seamless integration of backend logic with frontend design. [2]
- Ensure compatibility across different platforms while maintaining high performance. [2]

By leveraging JavaFX, the application achieves a user-friendly interface, making it easier for guests, hosts, and administrators to interact with the system effectively.

System Overview:

Application Scope

In the context of Express BnB, application scope plays a pivotal role in defining how resources, functionalities, and components interact and persist throughout the application's lifecycle. This ensures an optimized and seamless experience for users while maintaining a scalable and robust architecture.[3]

The application manages shared resources like properties, accommodations, and booking states centrally, allowing these components to be accessed by different user types—guests, hosts, and administrators—based on their specific needs. For example, accommodations remain accessible to guests for searching and booking while being dynamically updated by hosts based on availability. Notifications and other shared data persist across sessions and are universally accessible, enabling real-time updates for all users.

Key Highlights:

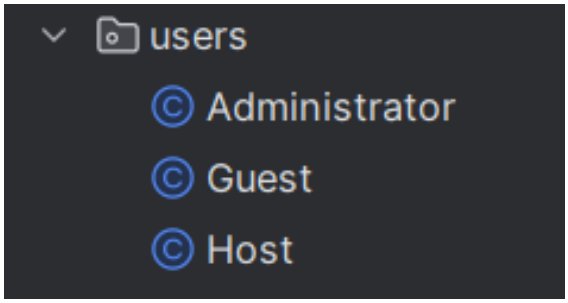
- **Lifecycle Management**: Objects such as bookings and user sessions are managed for optimal persistence, ensuring they are active only as long as necessary. For instance, booking details remain available to both hosts and guests until they are completed or cancelled.
- **Visibility and Accessibility**: Resources like property and booking information are modularly designed to be accessible across multiple components, such as host dashboards, guest views, and administrative tools, while maintaining encapsulation.
- **Scalability and Efficiency**: The system's architecture is designed to handle increased user demands. Key features, such as layered state transitions and notification management, are implemented with scalability in mind to ensure the platform grows with its user base.

Express BnB's application scope provides a foundation for efficient resource management, streamlined interactions, and future scalability, aligning with the project's goals of usability, maintainability, and adaptability.[3] This scope ensures a cohesive integration of the backend, GUI, and user interactions, delivering a consistent and engaging user experience.[3] Express BnB is designed to connect travellers with property hosts. The application supports customizable accommodations, catering to a broad spectrum of user preferences and budgets.

User Demographics

Express BnB caters to three primary user groups:

- **Travelers (Guests):** Guests use the platform to search for, filter, and book accommodations that suit their travel needs. They can also leave reviews to share their experiences.
- **Property Owners (Hosts):** Hosts manage accommodation listings, handle bookings, and receive notifications about guest activities, including reviews and booking requests.
- **Administrators:** Administrators maintain system functionality, manage user accounts, and oversee platform compliance, ensuring a smooth experience for all users.



User Stories

An integral component of requirements engineering, is a systematic process of defining, documenting, and maintaining the requirements of a system.[4] Requirements engineering bridges the gap between user needs and technical implementation by capturing the desired functionalities and constraints of an application. This process ensures that the end product aligns with user expectations while meeting technical feasibility.[5]

In the development of Express BnB, user stories serve as a guide to understanding the perspectives of different user types from the very beginning. By focusing on their needs and goals, these stories establish a clear scope for the project, setting the foundation for core functionalities required to create a usable and effective application.

Purpose of User Stories

User stories contribute significantly to requirements engineering by translating high-level objectives into detailed requirements. They help identify functional requirements, such as the ability to book accommodations, and non-functional requirements, such as scalability and usability.[4][5] By doing so, they ensure that the system is both capable of meeting user needs and maintaining performance, security, and reliability.

The following user stories outline the specific features and functionalities that the users expect from the system:

Guest Stories

- **As a guest**, I want to leave a review for an accommodation after my stay so that other guests can benefit from my experience.
- **As a guest**, I want to search for available accommodations by location, type, and price so that I can find a suitable place to book quickly.
- **As a guest**, I want to book an available accommodation for specific dates so that I can ensure my stay is reserved.
- **As a guest**, I want to cancel a booking if my plans change so that I won't incur unnecessary charges.
- **As a guest**, I want to receive notifications about my bookings, such as confirmation or cancellation so that I stay informed of any changes or updates.
- **As a guest**, I want to view all my bookings so that I can keep track of my scheduled stays.
- **As a guest**, I want to filter accommodations based on specific facilities, like a kitchen or pet-friendly options So that I can find exactly what I need for my stay

Host Stories

- **As a host**, I want to read reviews left by guests for my accommodation so that I can improve my offerings based on feedback.
- **As a host**, I want to list a new accommodation with details such as type, price, and availability so that guests can book my property.
- **As a host**, I want to view and manage bookings for my listed accommodations so that I can stay updated on my property's bookings and manage my schedule.
- **As a host**, I want to update the availability of my accommodations so that I can accurately reflect which dates are open for booking.
- **As a host**, I want to receive notifications when a booking is made or cancelled so that I can keep track of changes to my property's bookings.

Administrator Stories

- **As an administrator**, I want to monitor and address feedback or complaints so that user satisfaction is maintained.
- **As an administrator**, I want to view overall activity and statistics within the system so that I can ensure the platform operates smoothly.
- **As an administrator**, I want to add, update, or remove users (hosts and guests) so that the system remains secure and up-to-date.
- **As an administrator**, I want to respond to account deletion requests from users so that user data is removed as per their request.

Additionally, these user stories define core functionalities by breaking down the needs of each user type into actionable usability goals. Usability is a primary principle in application design, as it directly impacts user satisfaction. These user stories ensure that the interface and interactions are intuitive, efficient and accessible. As a result, it facilitates valuable communication between important stakeholders across the development process.

The Design Paradigms

One of the key paradigms that create a concrete foundation for this project is Object-oriented programming (OOP). Organising software design around data, or objects, rather than functions and logic is a principle that is fully implemented within this design.[6] It emphasizes four key principles: abstraction, encapsulation, inheritance, and polymorphism, which provide a blueprint for modular, scalable, and maintainable software development.[6] In the case of Express BnB, these principles were integral to the design and implementation of the system, ensuring a user-friendly and robust platform for admins, guests and hosts.

Key Object-Oriented Programming (OOP) Principles

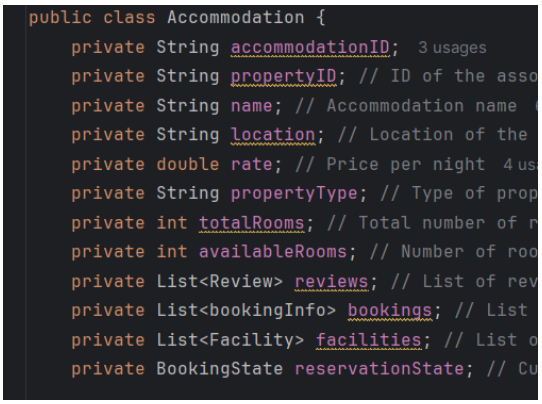
1. **Abstraction**
   Abstraction involves hiding the implementation details of a class and exposing only the necessary functionalities to the user.[6] [7] This simplifies the user's interaction with the system while managing complex logic internally. For instance, the BookingState interface abstracts the details of different booking states (e.g., PendingState, ConfirmedState) while providing consistent methods like reserve(), cancel(), and complete(). This abstraction allows accommodations to seamlessly transition between states without revealing the underlying logic.

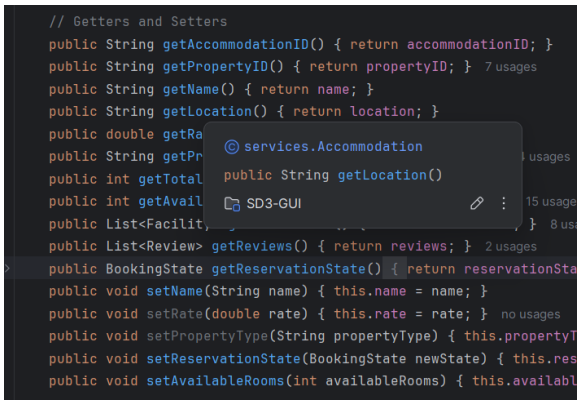
   - **Example**: The BookingState interface ensures that each state (e.g., PendingState, ConfirmedState) implements consistent behaviors without exposing the internal state management logic.

```
1   package models.States;
2
3   import services.Accommodation;
4
5   public interface BookingState {  14 usages  6 implementations
6       boolean checkAvailability();  3 usages  6 implementations
7       void reserve(bookingInfo booking);  1 usage  6 implementations
8       void cancel(bookingInfo booking);  6 implementations
9       void confirm(Accommodation accommodation, bookingInfo booking);  6 implementations
10      void cancel(Accommodation accommodation, bookingInfo booking);  6 implementations
11      void complete(Accommodation accommodation, bookingInfo booking);  1 usage  6 implementations
12  }
13
```

2. **Encapsulation**
   Encapsulation is the bundling of data (attributes) and methods that operate on that data within a single class, while restricting direct access to some of the object's components.[6][7] This is achieved using private access modifiers for class attributes and public getter and setter methods.

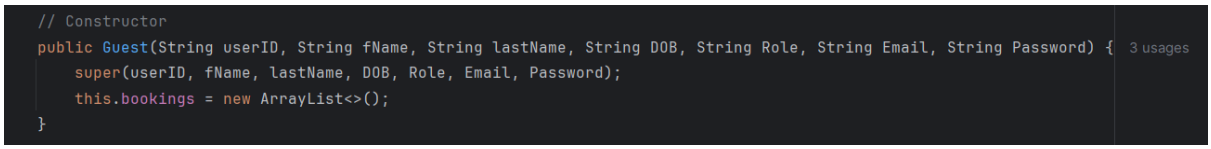
   - **Example**: In the Accommodation class, attributes like availableRooms and rate are encapsulated. Access to these attributes is controlled through methods like getAvailableRooms() and setRate(), ensuring data integrity.





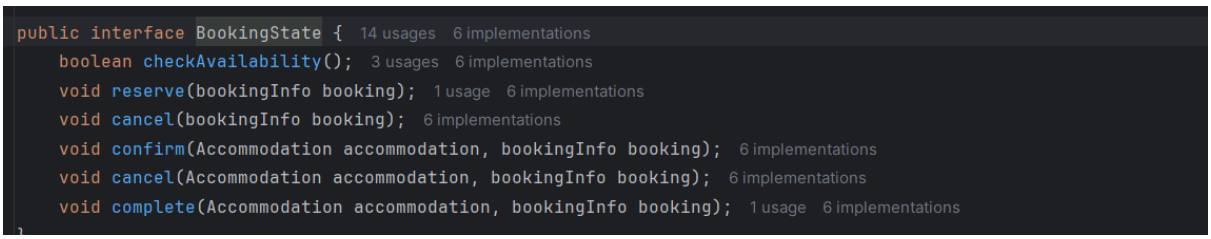
3. **Inheritance**
   Inheritance allows one class to inherit properties and methods from another, promoting code reuse and reducing redundancy. [6][7] In Express BnB, the User class serves as a base class for Guest, Host, and Administrator, enabling each subclass to share common attributes and methods while implementing unique behaviours.

   - **Example**: The Guest class inherits general user properties like userID and email from User while adding guest-specific functionalities such as addBooking() and writeReview().



4. **Polymorphism**
   Polymorphism allows objects of different classes to be treated as objects of a common superclass. It enables methods to process objects differently based on their actual type. [6][7] This is prominently used in Express BnB, where the BookingState interface allows methods like confirm() to behave differently based on the current state (e.g., PendingState vs. ConfirmedState).

   - **Example**: A BookingState objects dynamically invokes the appropriate state-specific implementation of methods such as complete() or cancel(), ensuring flexibility and extensibility.



The Role of Unified Modelling Language (UML)

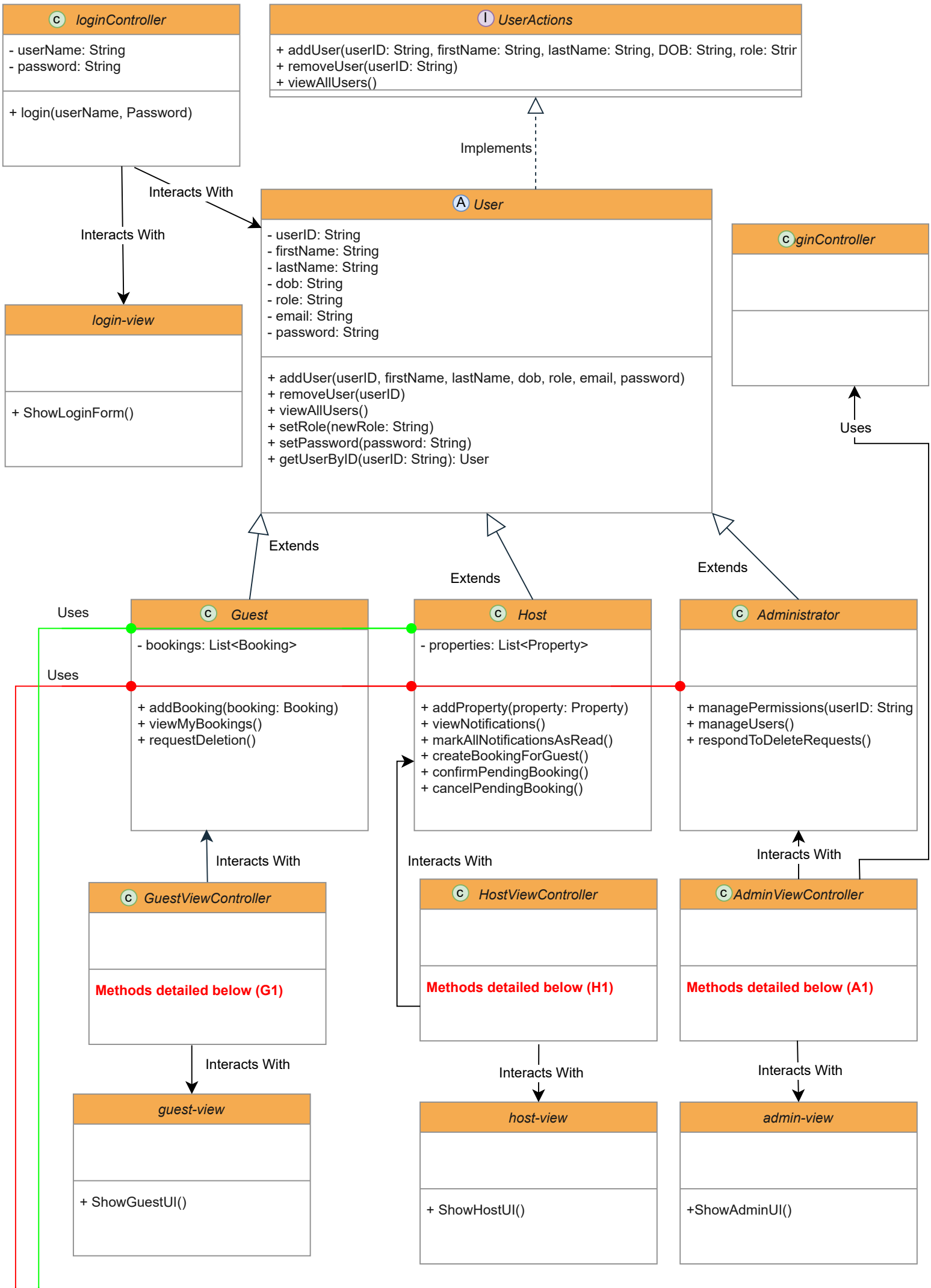
Another core programming paradigm that played a vital role in visualizing and designing the scalable and maintainable architecture of an object-oriented system was the implementation the unified modelling language (UML). These diagrams depict a clear relationship between classes including inheritance, associations, and dependencies, and provide a clear blueprint for implementation.[1] One example of this can be found when evaluating the relationship between the Accommodation class and the BookingState interface is depicted as a composition, ensuring that the accommodation's behaviour adapts dynamically to its current state.

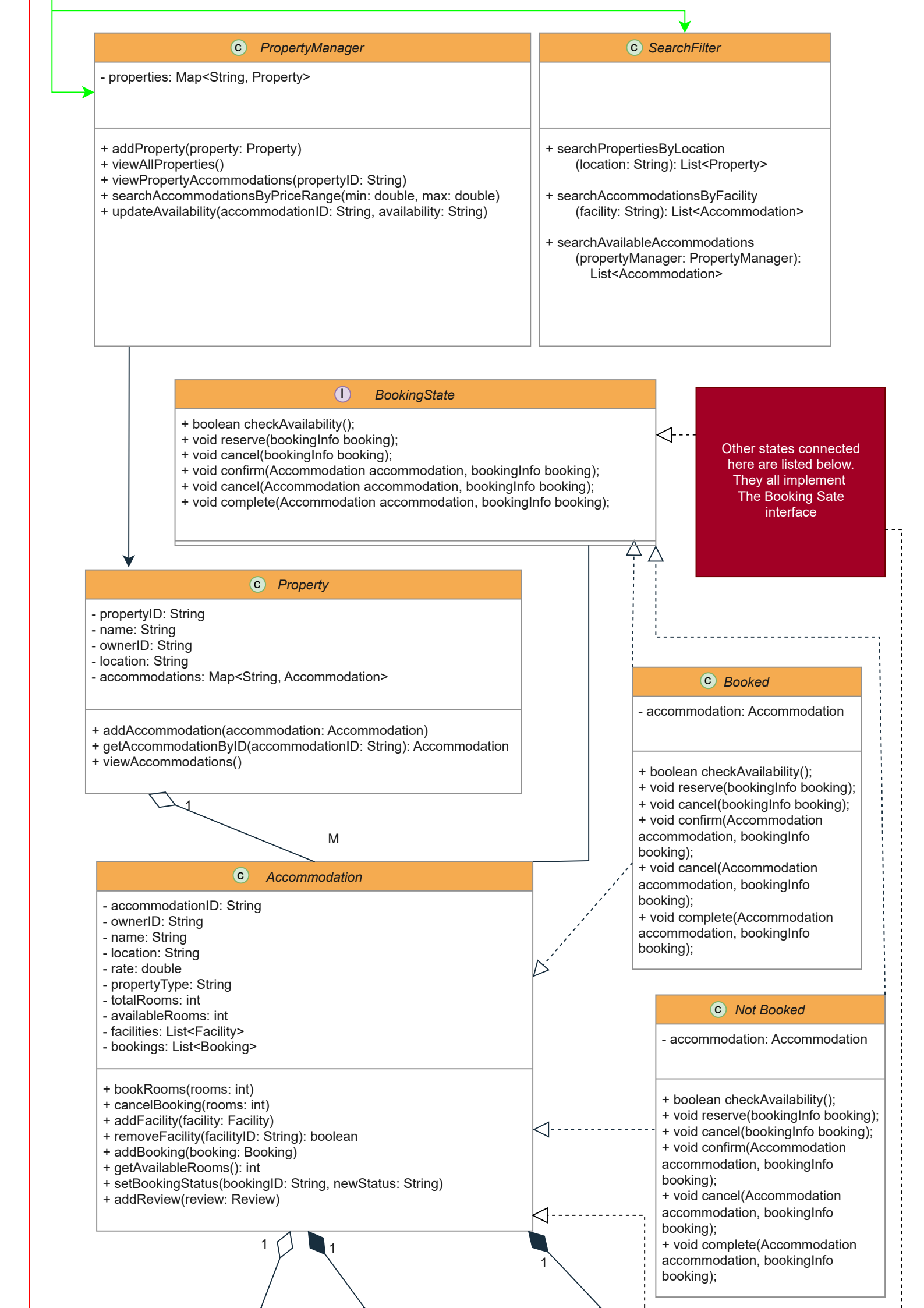
The Class Diagram:

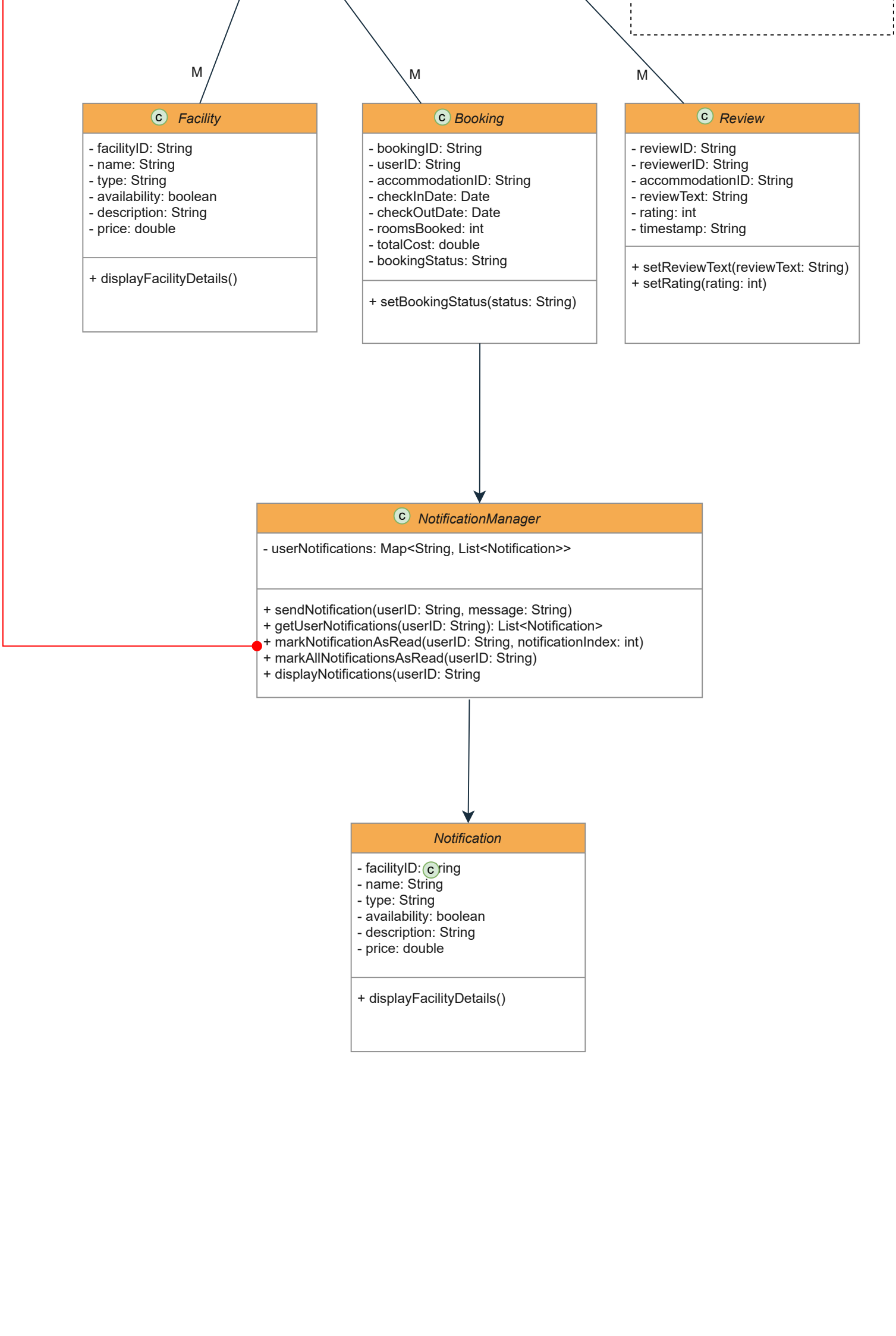
The design of classes in Express BnB reflects a meticulous application of object-oriented principles to model real-world entities with distinct responsibilities. At the core lies the abstract User class, which encapsulates common attributes such as userID, firstName, and email. Specialized subclasses (Guest, Host, and Administrator) inherit from this base to implement functionalities tailored to their specific roles. For instance, the Guest class facilitates reservation management and review submission, while the Host class enables accommodation oversight and booking management. The Accommodation class represents lodging entities with attributes such as rate and availableRooms, and the bookingInfo class encapsulates the details of individual bookings, including lifecycle states and transitions, via methods like confirm() and cancel().[1]

Constructors are employed to ensure objects are instantiated with consistent and well-defined attributes. Express Bnb, rigorously maintains controlled access to class attributes, using secure getter and setter methods encapsulated into their classes thus safeguarding the integrity of available data. [1]

Each class defines behaviour through a range of methods, from fundamental accessors to sophisticated state-transition mechanisms. Furthermore, a toString() method is implemented across key classes, such as bookingInfo, to facilitate clear, human-readable representations of object states. This design not only ensures a robust, modular architecture but also aligns with best practices in software engineering, enhancing maintainability and scalability. [1]


loginController
- userName: String
- password: String
+ login(userName, Password)
UserActions
+ addUser(userID: String, firstName: String, lastName: String, DOB: String, role: Strir
+ removeUser(userID: String)
+ viewAllUsers()
Implements
Interacts With
Interacts With
User
- userID: String
- firstName: String
- lastName: String
- dob: String
- role: String
- email: String
- password: String
+ addUser(userID, firstName, lastName, dob, role, email, password)
+ removeUser(userID)
+ viewAllUsers()
+ setRole(newRole: String)
+ setPassword(password: String)
+ getUserByID(userID: String): User
login-view
+ ShowLoginForm()
ginController
Uses
Extends
Extends
Extends
Uses
Uses
Guest
- bookings: List<Booking>
+ addBooking(booking: Booking)
+ viewMyBookings()
+ requestDeletion()
Host
- properties: List<Property>
+ addProperty(property: Property)
+ viewNotifications()
+ markAllNotificationsAsRead()
+ createBookingForGuest()
+ confirmPendingBooking()
+ cancelPendingBooking()
Administrator
+ managePermissions(userID: String
+ manageUsers()
+ respondToDeleteRequests()
Interacts With
Interacts With
Interacts With
GuestViewController
Methods detailed below (G1)
HostViewController
Methods detailed below (H1)
AdminViewController
Methods detailed below (A1)
Interacts With
Interacts With
Interacts With
guest-view
+ ShowGuestUI()
host-view
+ ShowHostUI()
admin-view
+ShowAdminUI()


PropertyManager
- properties: Map<String, Property>
+ addProperty(property: Property)
+ viewAllProperties()
+ viewPropertyAccommodations(propertyID: String)
+ searchAccommodationsByPriceRange(min: double, max: double)
+ updateAvailability(accommodationID: String, availability: String)
SearchFilter
+ searchPropertiesByLocation (location: String): List<Property>
+ searchAccommodationsByFacility (facility: String): List<Accommodation>
+ searchAvailableAccommodations (propertyManager: PropertyManager): List<Accommodation>
BookingState
+ boolean checkAvailability();
+ void reserve(bookingInfo booking);
+ void cancel(bookingInfo booking);
+ void confirm(Accommodation accommodation, bookingInfo booking);
+ void cancel(Accommodation accommodation, bookingInfo booking);
+ void complete(Accommodation accommodation, bookingInfo booking);
Other states connected here are listed below. They all implement The Booking Sate interface
Property
- propertyID: String
- name: String
- ownerID: String
- location: String
- accommodations: Map<String, Accommodation>
+ addAccommodation(accommodation: Accommodation)
+ getAccommodationByID(accommodationID: String): Accommodation
+ viewAccommodations()
1
M
Booked
- accommodation: Accommodation
+ boolean checkAvailability();
+ void reserve(bookingInfo booking);
+ void cancel(bookingInfo booking);
+ void confirm(Accommodation accommodation, bookingInfo booking);
+ void cancel(Accommodation accommodation, bookingInfo booking);
+ void complete(Accommodation accommodation, bookingInfo booking);
Accommodation
- accommodationID: String
- ownerID: String
- name: String
- location: String
- rate: double
- propertyType: String
- totalRooms: int
- availableRooms: int
- facilities: List<Facility>
- bookings: List<Booking>
+ bookRooms(rooms: int)
+ cancelBooking(rooms: int)
+ addFacility(facility: Facility)
+ removeFacility(facilityID: String): boolean
+ addBooking(booking: Booking)
+ getAvailableRooms(): int
+ setBookingStatus(bookingID: String, newStatus: String)
+ addReview(review: Review)
Not Booked
- accommodation: Accommodation
+ boolean checkAvailability();
+ void reserve(bookingInfo booking);
+ void cancel(bookingInfo booking);
+ void confirm(Accommodation accommodation, bookingInfo booking);
+ void cancel(Accommodation accommodation, bookingInfo booking);
+ void complete(Accommodation accommodation, bookingInfo booking);
1
1
1


M
M
M
Facility
- facilityID: String
- name: String
- type: String
- availability: boolean
- description: String
- price: double
+ displayFacilityDetails()
Booking
- bookingID: String
- userID: String
- accommodationID: String
- checkInDate: Date
- checkOutDate: Date
- roomsBooked: int
- totalCost: double
- bookingStatus: String
+ setBookingStatus(status: String)
Review
- reviewID: String
- reviewerID: String
- accommodationID: String
- reviewText: String
- rating: int
- timestamp: String
+ setReviewText(reviewText: String)
+ setRating(rating: int)
NotificationManager
- userNotifications: Map<String, List<Notification>>
+ sendNotification(userID: String, message: String)
+ getUserNotifications(userID: String): List<Notification>
+ markNotificationAsRead(userID: String, notificationIndex: int)
+ markAllNotificationsAsRead(userID: String)
+ displayNotifications(userID: String
Notification
- facilityID: String
- name: String
- type: String
- availability: boolean
- description: String
- price: double
+ displayFacilityDetails()

Other States

PendingState

- accommodation: Accommodation

+ boolean checkAvailability();
+ void reserve(bookingInfo booking);
+ void cancel(bookingInfo booking);
+ void confirm(Accommodation accommodation, bookingInfo booking);
+ void cancel(Accommodation accommodation, bookingInfo booking);
+ void complete(Accommodation accommodation, bookingInfo booking);

ConfirmedState

- accommodation: Accommodation

+ boolean checkAvailability();
+ void reserve(bookingInfo booking);
+ void cancel(bookingInfo booking);
+ void confirm(Accommodation accommodation, bookingInfo booking);
+ void cancel(Accommodation accommodation, bookingInfo booking);
+ void complete(Accommodation accommodation, bookingInfo booking);

CanceledState

+ boolean checkAvailability();
+ void reserve(bookingInfo booking);
+ void cancel(bookingInfo booking);
+ void confirm(Accommodation accommodation, bookingInfo booking);
+ void cancel(Accommodation accommodation, bookingInfo booking);
+ void complete(Accommodation accommodation, bookingInfo booking);

CompletedState

+ boolean checkAvailability();
+ void reserve(bookingInfo booking);
+ void cancel(bookingInfo booking);
+ void confirm(Accommodation accommodation, bookingInfo booking);
+ void cancel(Accommodation accommodation, bookingInfo booking);
+ void complete(Accommodation accommodation, bookingInfo booking);

G1,H1,A1 Details

A1

- populateRoleChoiceBox(): Void
- showAlert(String title, String message, Alert.AlertType alertType): Void
- deleteUserRequest(): Void
- updateNewUser(): Void
- addNewUser(): Void
- clearAddUserFields(): Void
- isValidDate(String date): Boolean
- isValidEmail(String email): Boolean
- populatePerformanceChart(): Void
+ adminSignOut(): Void
+ populateDeleteRequests(): Void
+ populateUserIDs(): Void
+ populateOverviewChart(): Void
+ initialize(): Void

G1

- showAlert(String title, String message, Alert.AlertType alertType): Void

+ cancelBooking(): Void
+ GuestSignOut(): Void
+ makeBooking(): Void
+ makeDeleteRequest(): Void
+ markAsRead(): Void
+ populateNotifications(User activeUser): Void
+ updatePerformanceTracker(): Void

+ viewAccommodations(PropertyManagerCommand propertyManager): Void

+ viewBookings(User activeUser): Void
+ writeReview(): Void

H1

+ hostSignOut(): Void
+ makeBooking(): Void
+ updateAvailability(): Void
+ markAsRead(): Void
+ completeCurrentBooking(): Void
+ confirmPendingBooking(): Void
+ cancelPendingBooking(): Void
+ cancelCurrentBooking(): Void
+ manageFacility(): Void
+ makeDeleteRequest(): Void
+ updatePerformanceTracker(): Void

+ populateCompletedBookings(User activeUser, PropertyManagerCommand propertyManager): Void

+ populateMyAccommodations(User activeUser, PropertyManagerCommand propertyManager): Void

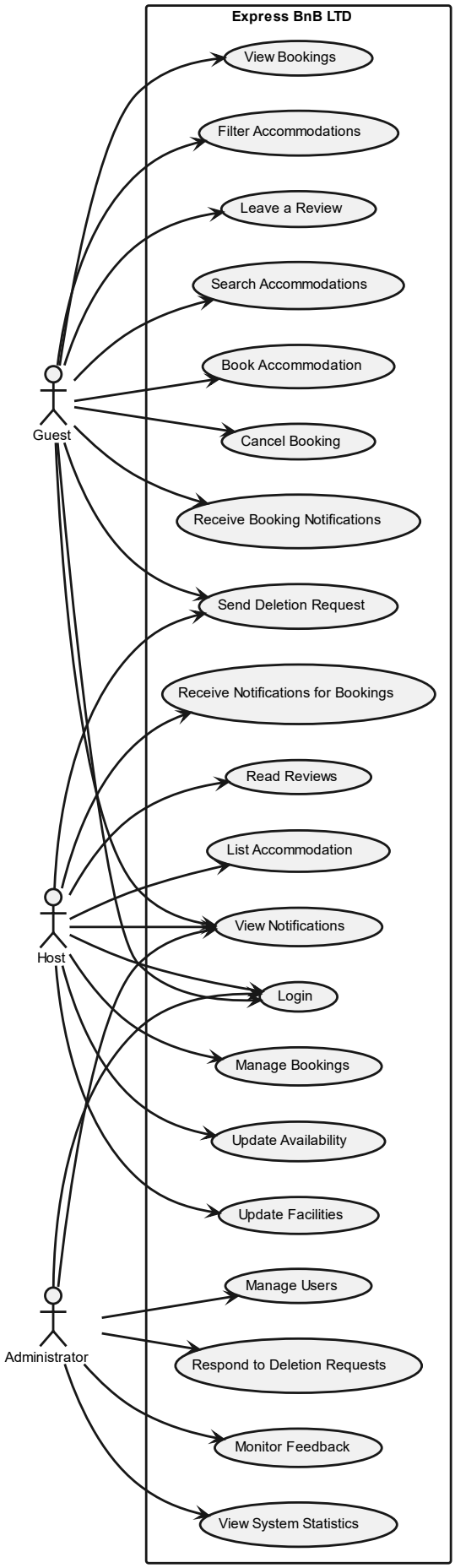
+ viewBookings(User activeUser, PropertyManagerCommand propertyManager): Void

+ listNewAccommodation(User activeUser): Void

+ populateNotifications(User activeUser): Void

+ populateReviews(User activeUser, PropertyManagerCommand propertyManager): Void

+ populatePendingBookings(User activeUser, PropertyManagerCommand propertyManager): Void



Use Case Diagram

The Use Case Diagram highlights the interactions between various users of the system and the primary functionalities they can perform. It illustrates the relationships between Guests, Hosts, and Administrators with the system's core operations, such as managing bookings, accommodations, and notifications. Each use case corresponds to user stories we identified earlier, ensuring that all user requirements are adequately addressed.[1] For instance:

- Guests can perform actions like searching for accommodations, making or cancelling bookings, and leaving reviews.
- Hosts manage accommodations and bookings while receiving guest feedback.
- Administrators oversee system-wide operations like user management and complaint resolution.

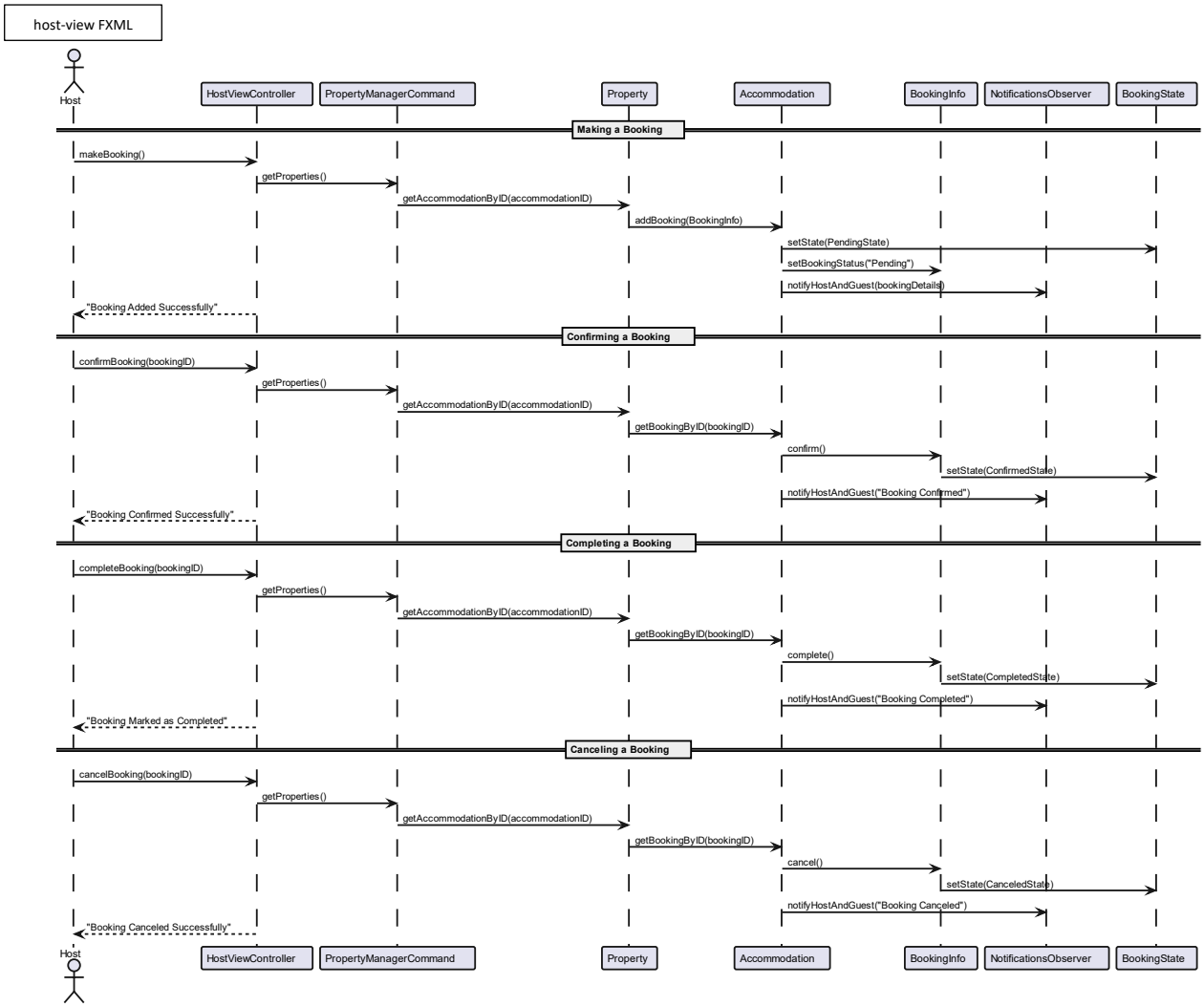
This diagram emphasizes the inclusivity of all user roles and ensures the application's design aligns with stakeholder expectations.

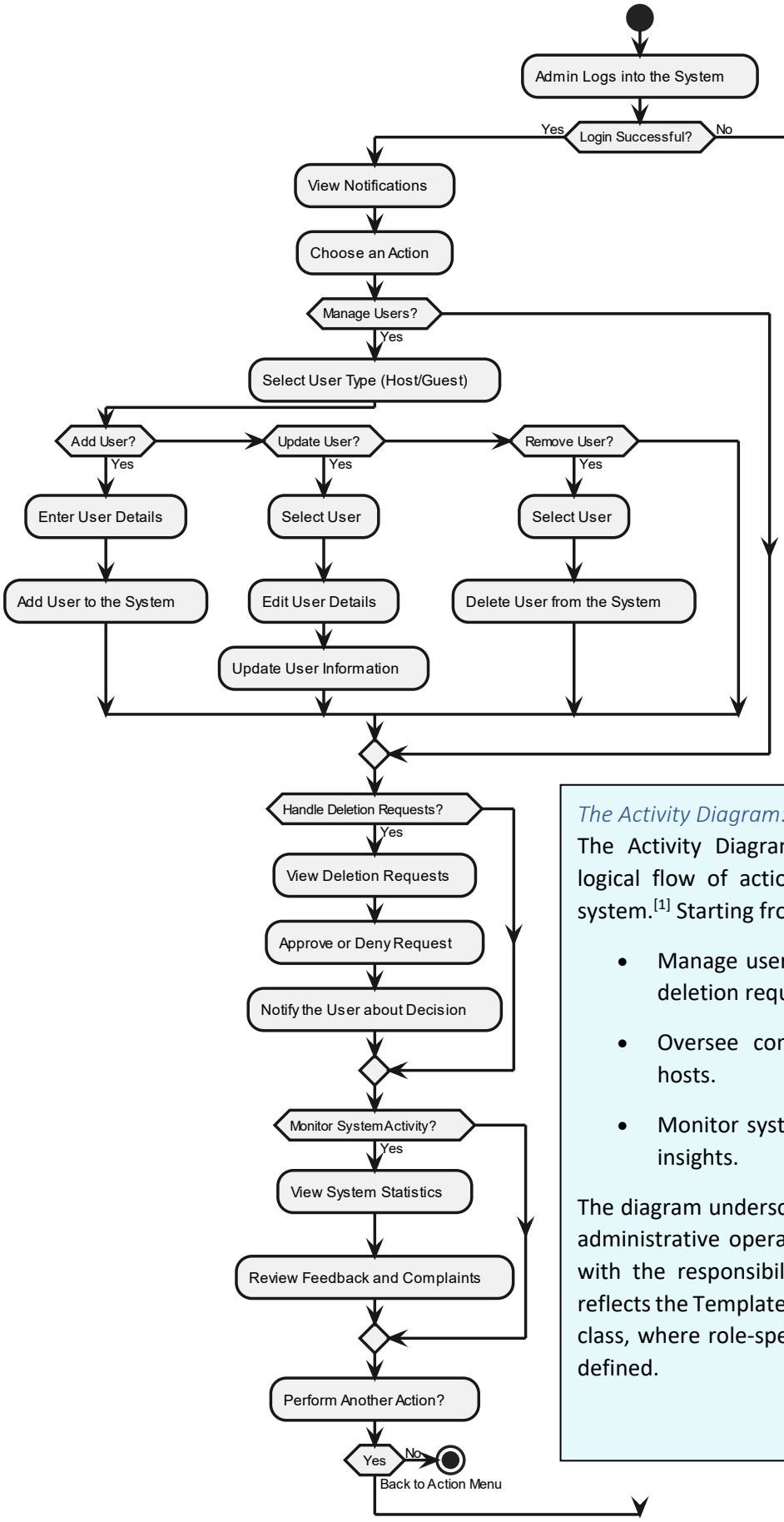
Sequence Diagram

The Sequence Diagram focuses on the interactions and message flow between objects during a Host's management of bookings, such as making, confirming, completing, or canceling a booking. By referencing the Host, Accommodation, BookingState, and NotificationsObserver classes, the diagram illustrates:

- How state transitions (e.g., from "Pending" to "Confirmed") are triggered by host actions.
- The dynamic notifications sent to Guests and Hosts when bookings are updated.
- The layered state management logic, showing the interplay between accommodation-level and property-level states.

This diagram highlights the dynamic behaviour of the system and the seamless integration of various design patterns, particularly the Layered State Pattern and Observer Pattern, to ensure real-time updates and consistency.[8]





The Activity Diagram:

The Activity Diagram for the Administrator illustrates the logical flow of actions performed by administrators in the system.[1] Starting from logging in, administrators can:

- Manage users by adding, removing, or responding to deletion requests.
- Oversee complaints or feedback from guests and hosts.
- Monitor system activity and statistics for operational insights.

The diagram underscores the sequential yet flexible nature of administrative operations, ensuring that the workflow aligns with the responsibilities of this critical user role. [1] It also reflects the Template Method Pattern implemented in the User class, where role-specific actions like administrative tasks are defined.

Advanced Features & Design Patterns

The development of Express BnB integrates advanced features and a variety of design patterns to enhance the application's scalability, modularity, and maintainability. Among these, the Layered State Pattern, uniquely developed for this project, plays a pivotal role in managing dynamic booking and property states, distinguishing the application from conventional platforms.

Additional Features:

The platform goes beyond standard booking functionalities to offer an enriched user experience, including:

- Facilities and Equipment: Accommodations come with dynamic features such as gyms, pools, and parking, enabling users to filter properties based on their preferences. These facilities are modeled using composition, allowing properties to aggregate features without affecting their core structure.
- Searchability: Java Streams power robust filtering capabilities, allowing users to search accommodations by location, price range, or specific facilities seamlessly.

Composition in System Modelling:

Composition is extensively employed to model relationships between objects, ensuring modularity and reusability. This ensures a decoupled and adaptable system design, allowing independent evolution of components.[9]

- Properties and Accommodations: Properties aggregate multiple accommodations, providing flexibility for expansion.
- Facilities: Facilities are dynamically associated with accommodations, making it easy to add or modify features without altering the main structure.

Design Patterns Used

Express BnB incorporates four key design patterns, with the Layered State Pattern being a unique innovation for this project:

Layered State Pattern (Unique to This Project)

Overview: This pattern manages two distinct levels of state:

- Property Level: Determines whether a property is "Available" or "Fully Booked" based on the states of its accommodations.
- Accommodation Level: Implements a four-state system (Pending, Confirmed, Canceled, Completed) for granular booking management.

A summarized example of its Implementation in Java:

```java
class Accommodation {
    private BookingState currentState;
    public void setBookingState(BookingState state) {
        this.currentState = state;
    }
    public void confirmBooking(bookingInfo booking) {
        currentState.confirm(this, booking);
    }
}
```

Benefits:

- Ensures context-aware booking logic with seamless state transitions.
- Supports dynamic behaviour changes across property and accommodation levels.

Observer Pattern

Overview: Manages notifications for guests and hosts dynamically.[10]

Implementation:

```java
NotificationsObserver notificationsManager = NotificationsObserver.getInstance();
notificationsManager.sendNotification(guestID, "Your booking has been confirmed.");
```

Benefits:

- Decouples notification logic from core functionalities.
- Scales effortlessly as the platform grows.

Strategy Pattern

Overview: Enables interchangeable filtering algorithms for accommodations. [10]

Implementation:

```java
List<Accommodation> filtered = searchFilterStrategy.searchAccommodationsByPriceRange(minPrice, maxPrice, propertyManager);
```

Benefits:

- Facilitates the addition of new filtering strategies without modifying existing logic.
- Enhances testing and debugging by isolating filtering methods.

Template Method Pattern

Overview: Defines a base template for user actions, while subclasses (Guest, Host, Administrator) implement specific behaviours. [10]

Implementation:

```java
abstract class User {
    public void login(String username, String password) {
        validateCredentials(username, password);
        postLoginActions();
    }
    protected abstract void postLoginActions();
}
class Guest extends User {
    @Override
    protected void postLoginActions() {
        System.out.println("Guest-specific actions.");
    }
}

```

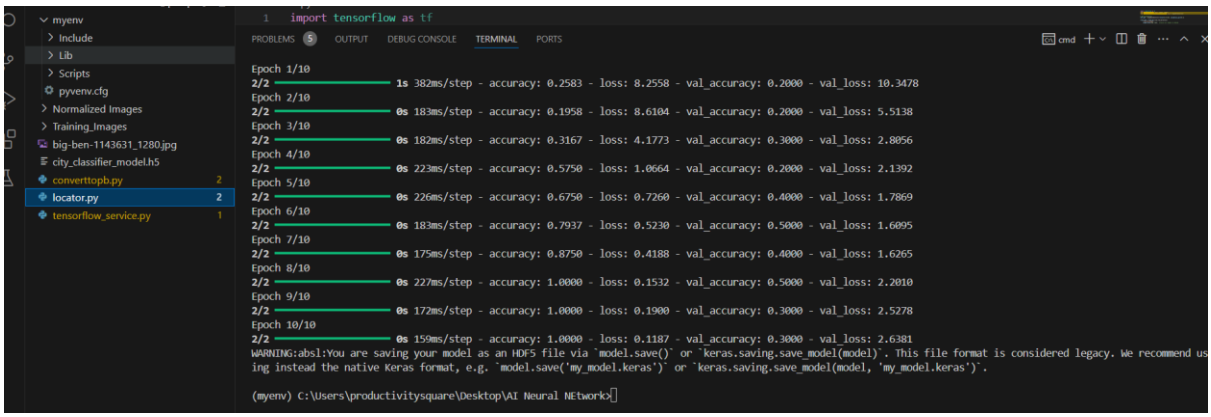
Benefits:

- Promotes code reuse by centralizing common actions.
- Ensures consistent behaviors across different user roles.

Convolutional Neural Network Design & Implementation

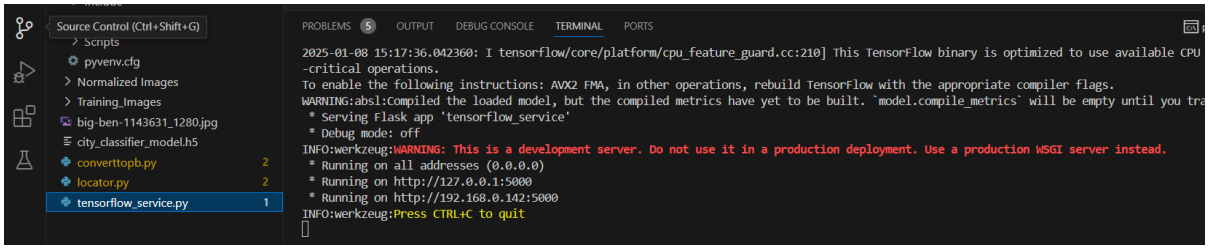
Creating and training the CNN Model

The project was extended with the development of a Convolutional Neural Network (CNN) using TensorFlow in Python. The model was designed to classify images into specific categories. To optimize training, we preprocessed the dataset by resizing all images to a uniform size and normalizing pixel values to a 0-1 range. This normalization step was crucial for ensuring faster convergence during training. The model architecture consisted of convolutional layers for feature extraction and fully connected layers for classification. Once trained, the model was saved in a format suitable for deployment (city_classifier_model.h5).



Establishing a Client-Server Communication

To integrate the AI model with the Java-based application, we implemented a client-server architecture. A server was developed in Python using Flask to handle image file processing and make predictions using the trained TensorFlow model. The Java application acts as a client, transferring images to the Python server over a local network. This setup enabled seamless communication between the Java frontend and the Python backend, ensuring the AI model could be utilized effectively.



Managing Dependencies with Virtual Environments

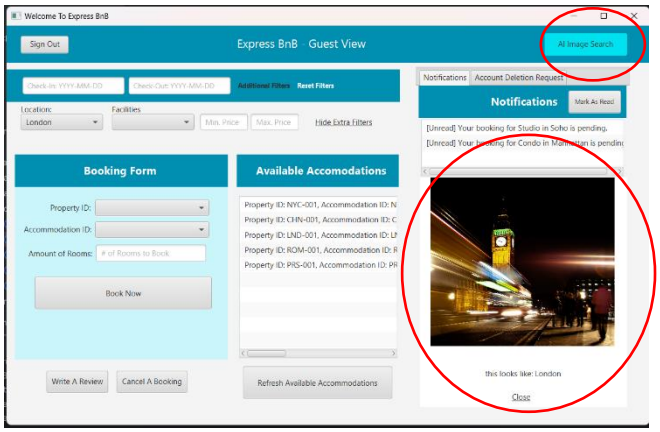
To manage the Python server's dependencies, we created a virtual environment. This approach ensured that all required libraries, including TensorFlow and Flask, were isolated and could be installed without affecting the global Python environment. The virtual environment also simplified deployment by ensuring that the server's dependencies were consistent across different systems.

```
(myenv) C:\Users\productivitysquare\Desktop\AI Neural NEtwork>
```

Resolving File Path Compatibility Issues

Initially, we attempted to integrate the systems by sharing file paths between the Java application and the Python server. However, this posed significant challenges due to differences in how Java and Python handle file path formats. Transitioning to a client-server architecture resolved these issues by allowing the Java application to transfer image data directly to the server, bypassing the need for shared paths.

Brief Explanation of How the AI Integration Works



The system allows users to perform an AI-driven image search to determine the location of a landmark. When the user clicks the "AI Image Search" button, a file explorer opens, enabling the user to select an image file from their device.

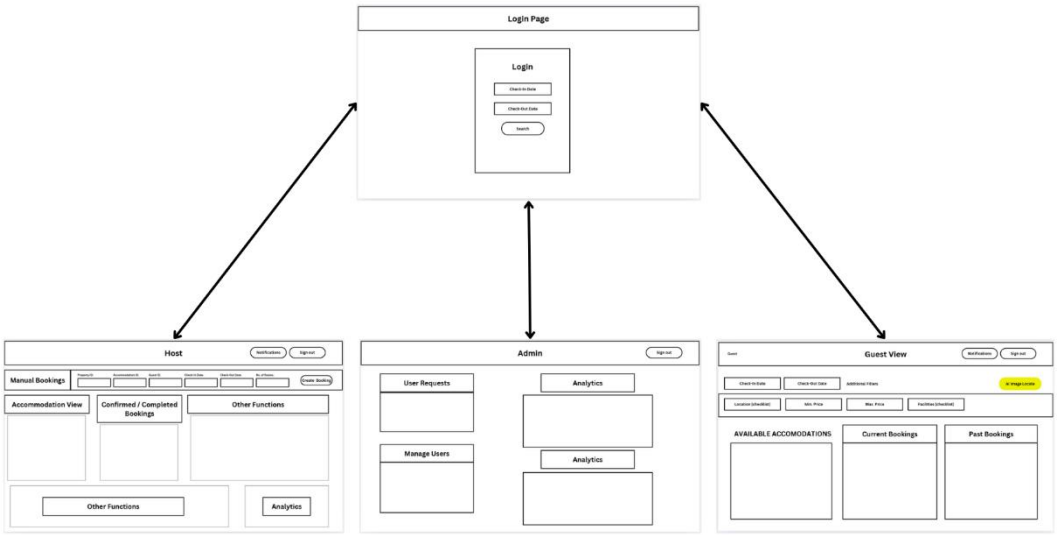
Once an image is selected, it is sent to the AI model, which processes the image to identify the location of the landmark depicted.

The AI model then determines the most likely location and sends the result back to the application. The identified location is automatically selected in the location choice box, streamlining the process for the user. Based on the selected location, the system fetches and displays a list of available accommodations in that area, providing a seamless and efficient user experience.

GUI Design and Implementation:

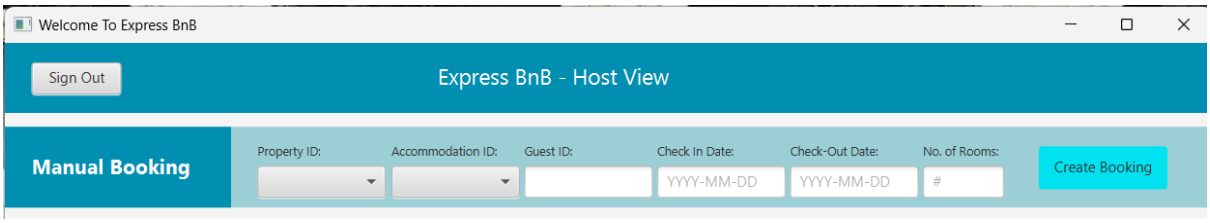
Wireframe and Design Rationale

The wireframe for the Express BnB application was crafted to ensure simplicity and ease of navigation for diverse users. Each screen was designed with a clear layout, featuring distinct sections for searching accommodations, managing bookings, and viewing notifications. The rationale behind this design was to minimize user confusion by structuring functionalities in an intuitive manner, aligned with the user stories defined during requirements analysis. The visual flow allows guests to quickly find and book accommodations, hosts to manage their properties, and administrators to oversee platform activity efficiently.[11]



Implementation of Basic GUI Elements

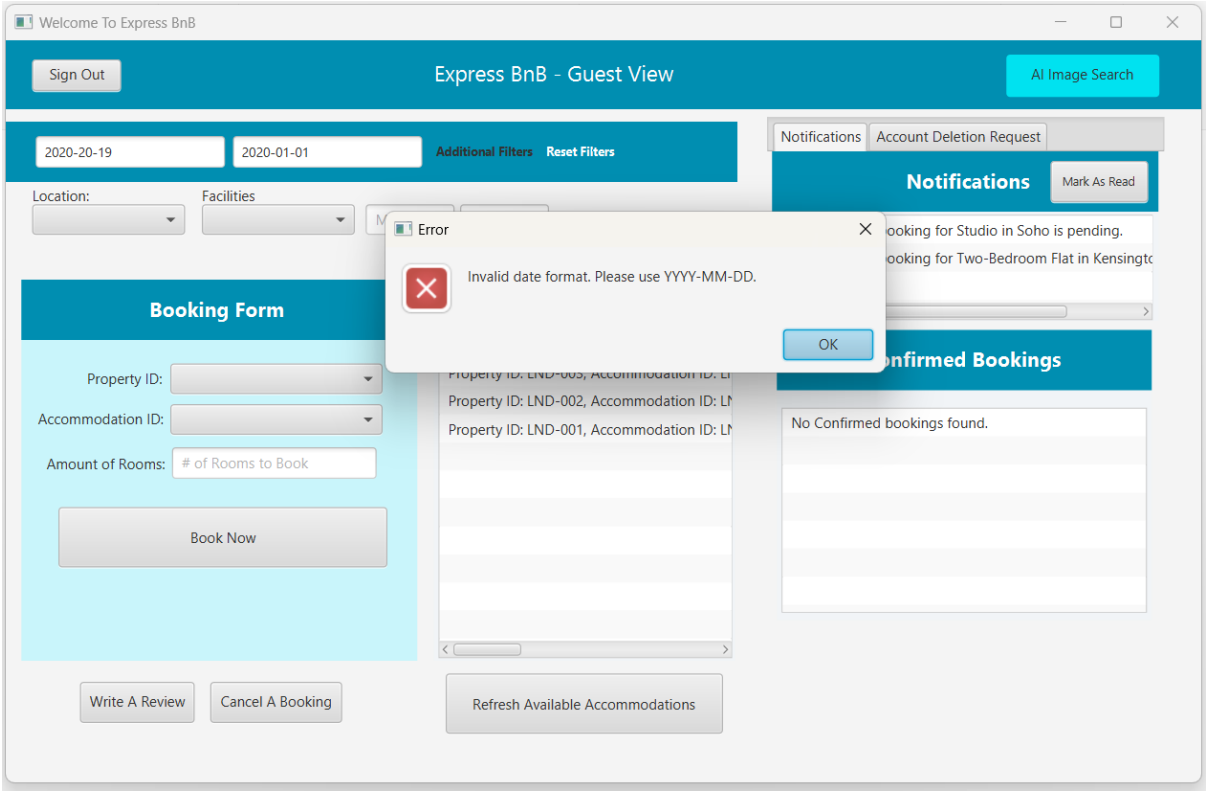
Using JavaFX**,** fundamental GUI elements such as text fields, tabs, buttons, combo boxes, and list views were implemented. These elements handle user interactions like entering search parameters, selecting booking options, and reviewing notifications. The implementation leverages event handlers to process user actions and provide real-time feedback, ensuring a smooth and interactive experience. For instance, list views dynamically update to show the latest booking statuses or search results.[2]



Advanced GUI Features

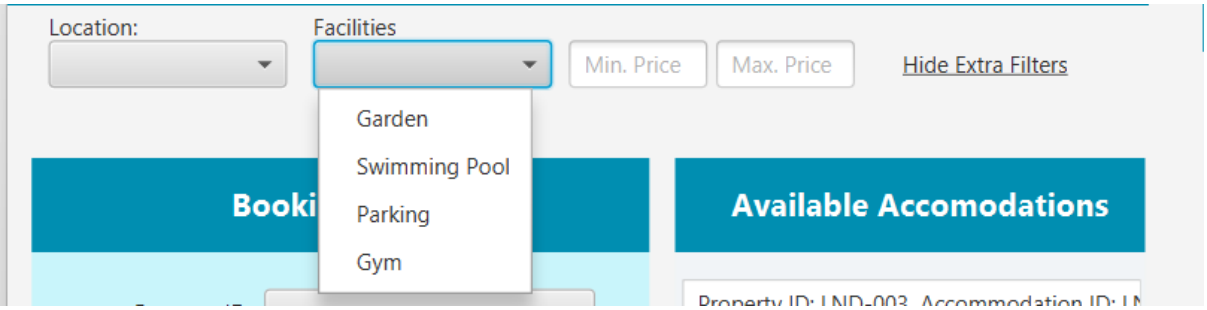
Advanced GUI features elevate the user experience by providing dynamic interactivity and responsiveness. For example:

- Real-time filtering and sorting of accommodations based on user inputs, such as location or price range, allow for a more personalized experience.
- Notification alerts pop up immediately when an event occurs, such as a booking confirmation or cancellation.
- The GUI incorporates user-friendly error handling, guiding users with appropriate prompts when invalid data is entered.



Java Streams Implementation in GUI

Java Streams were integrated into the GUI to enhance performance and efficiency in handling large data sets.[12] For instance, streams are used to filter accommodations based on multiple criteria, such as location, price, and facilities. The use of streams simplifies the backend logic by enabling concise, declarative code while ensuring optimal performance. This approach ensures that users can interact with the application seamlessly, even when processing extensive datasets or performing complex filtering operations.



The GUI design and implementation for Express BnB not only fulfil functional requirements but also provide an engaging and efficient interface, leveraging advanced JavaFX capabilities and modern programming techniques like streams.

Testing:
Testing Strategy

```
CancelationForm is now visible.
...........models.States.PendingState@43fc7f00models.States.notBookedState@2862541f
State updated to: CanceledState
Pending booking canceled: BOOK-001
...........models.States.CanceledState@3e9fc61dmodels.States.notBookedState@5a7ccd80
Notification sent to guest1: Your booking for Studio in Soho from 2024-03-01 to 2024-03-05 has been successfully canceled.
Notification sent to host1: Guest Alice canceled their booking for Studio in Soho from 2024-03-01 to 2024-03-05.
```

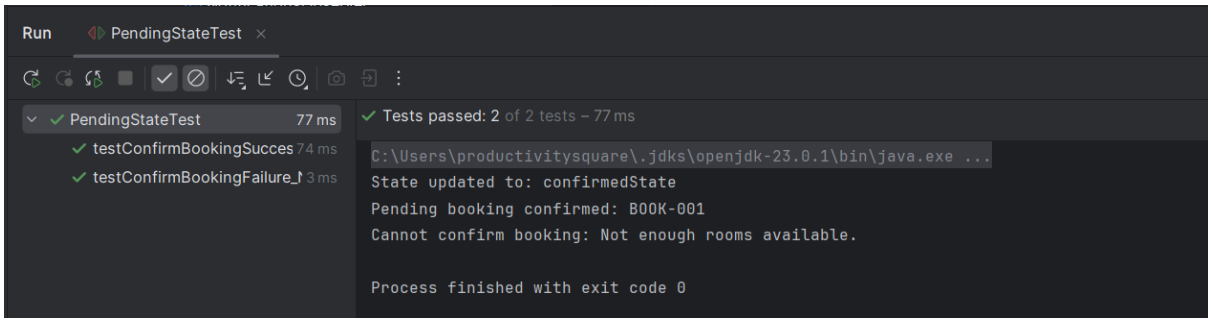
The testing process for Express BnB followed a structured and multi-layered strategy to ensure the application's functionality, reliability, and usability. The testing approach combined unit tests, integration tests**,** user acceptance testing, and Java debugging techniques to identify and resolve issues effectively.

Unit Tests

Unit testing focused on verifying the functionality of individual classes and methods. Key components such as the Guest, Host, and Accommodation classes underwent rigorous testing to ensure their methods performed as expected.[13] For example:

- **Booking Management**: Tests confirmed that bookings transitioned correctly between states (e.g., Pending → Confirmed → Completed) using the layered state pattern.
- **Filtering Logic**: Tests validated that accommodations were accurately filtered by location, price, and facilities.
- **Notifications**: The NotificationsObserver class was tested to ensure notifications were dispatched and received by the appropriate users.





Integration Tests

Integration testing validated the interaction between different components, ensuring that the system worked cohesively.[14] For example:

- **Guest and Accommodation Interaction**: Tests confirmed that when a guest booked an accommodation, availability updates were reflected, and notifications were sent to the host.
- **Host and Property Manager**: Tests verified that hosts could manage their properties and changes were seamlessly synchronized across the system.
- **GUI and Backend**: Tests ensured that GUI components, such as search filters and booking forms, interacted correctly with backend classes like PropertyManagerCommand and SearchFilterStrategy.

```
Filtering by location: Wimbledon and facilities: Swimming Pool
Performance tracker updated by GuestViewController
Filtering by location: Wimbledon and facilities: Gym
Performance tracker updated by GuestViewController
```

Integration tests were instrumental in identifying issues that emerged during component interactions, which were not evident in isolated unit tests.

User Acceptance Testing

User acceptance testing (UAT) ensured that the application met the expectations of real users. Testers acting as guests, hosts, and administrators evaluated the application's usability, functionality, and performance in real-world scenarios, such as: [15]

- Guests booking accommodations, canceling reservations, and leaving reviews.
- Hosts managing accommodations, confirming bookings, and monitoring notifications.
- Administrators addressing user deletion requests and overseeing system activity.

Feedback from UAT prompted iterative improvements, including enhanced error messages, better GUI responsiveness, and simplified workflows for booking and cancellation processes.

Debugging Techniques

Throughout the development process**,** Java debugging techniques played a crucial role in troubleshooting issues. The systematic use of:

- **Print Statements**: Strategic System.out.println statements were added to track object changes, method calls, and state transitions. [16]
- **Error Messages**: Informative error messages guided developers in pinpointing and resolving issues. [16]
- **Debugging Tools**: Java IDE debugging features, such as breakpoints and variable watchers, enabled step-by-step code execution analysis. [16]

These debugging practices complemented formal testing methods, ensuring robust issue resolution and code reliability.

Conclusion

The development of Express BnB marks a significant achievement in building a dynamic and scalable accommodation booking platform that effectively bridges the gap between travellers and property hosts. The project successfully demonstrates the power of robust software engineering principles, object-oriented programming, and design pattern integration to solve real-world problems.

Summary of Achievements

Key milestones of this project include:

- **Layered State Pattern Implementation:** A unique innovation for managing accommodations and their states, enabling seamless transitions between booking statuses while maintaining system integrity.
- **Advanced JavaFX GUI:** A user-friendly and visually engaging interface tailored to diverse user roles, enhancing usability and accessibility.
- **Sophisticated Programming Paradigms:** The application of object-oriented principles and design patterns, such as State, Observer, and Strategy, ensured modularity, scalability, and maintainability.
- **Feature-Rich System:** Core functionalities such as booking management, searchable accommodations, and real-time notifications significantly enhance the user experience for both guests and hosts.

Future Enhancements

While Express BnB achieves its current objectives, there is significant scope for future advancements:

- **Enhanced Search Capabilities:** Introduce advanced filtering mechanisms, including geolocation-based searches and machine learning-driven personalized recommendations.
- **Mobile Application Expansion:** Develop Android and iOS applications to increase accessibility and reach.
- **Data Analytics:** Offer hosts and administrators data visualization tools for booking trends, revenue analysis, and user behaviour insights.
- **Payment Integration:** Incorporate secure and efficient payment gateways for end-to-end booking experiences.
- **Multilingual Support:** Add language options to cater to global users and ensure inclusivity.